Type Classes via Dictionaries

Type Classes · Explicit Monad dictionary for Option · Build with Waqar

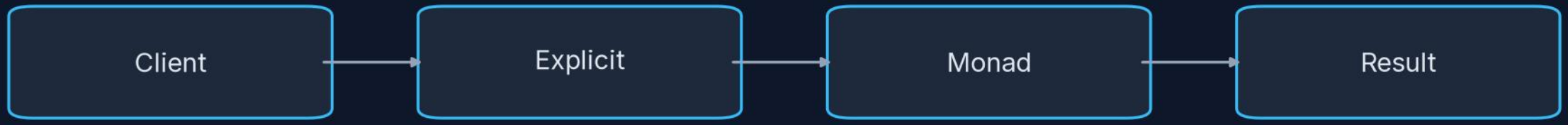
Today's Hook

- Problem: Explicit Monad dictionary for Option
- Pattern: Type Classes — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Explicit Monad dictionary for Option
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Type Classes: Explicit Monad dictionary for Option

Real-World Scenario

- Running example: Explicit Monad dictionary for Option
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.typeclass;

import com.buildwithwaqar.patterns.functional.typeclass.*;
import dev.zazr.control.Option;
/** Problem: map over Option via an explicit Monad dictionary (no HKTs). */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Typeclass/Monad problem: Option dictionary ===");
    var fa = OptionKind.lift(Option.some(21));
    System.out.println(OptionKind.lower(OptionKind.MONAD.map(fa, n -> n * 2)));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.typeclass;

import java.util.function.Function;

public interface Applicative<F> extends Functor<F> {
    <A> Kind<F, A> pure(A a);
    <A, B> Kind<F, B> ap(Kind<F, Function<A, B>> ff, Kind<F, A> fa);
}


package com.buildwithwaqar.patterns.functional.typeclass;

import java.util.function.Function;

public interface Monad<F> extends Applicative<F> {
    <A, B> Kind<F, B> flatMap(Kind<F, A> fa, Function<? super A, ? extends Kind<F, B>> f);

    @Override
    default <A, B> Kind<F, B> map(Kind<F, A> fa, Function<? super A, ? extends B> f) {
        return flatMap(fa, a -> pure(f.apply(a)));
    }
```

Solution (2)

```
    @Override
    default <A, B> Kind<F, B> ap(Kind<F, Function<A, B>> ff, Kind<F, A> fa) {
        return flatMap(ff, f -> map(fa, f));
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.typeclass;

import com.buildwithwaqar.patterns.functional.typeclass.*;
import dev.zazr.control.Option;
/** Problem: map over Option via an explicit Monad dictionary (no HKTs). */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Typeclass/Monad problem: Option dictionary ===");
    var fa = OptionKind.lift(Option.some(21));
    System.out.println(OptionKind.lower(OptionKind.MONAD.map(fa, n -> n * 2)));
  }
}
```

Walkthrough Output

```
=== Typeclass/Monad problem: Option dictionary ===
Some(42)
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Type Classes.

Related Patterns

- Contrast Type Classes with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Type Classes → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.